

DS Practical Important

1)Data Frame and Data Preprocessing.

```
# 📦 Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# ◇ Step 1: Load CSV File
df = pd.read_csv("data.csv") # 📄 Change the file name
print("Loaded Data:\n", df.head())
print("CSV Info:", df.info(), "\n")
print("CSV Description:\n", df.describe(), "\n")

# ◇ Step 2: Select numeric columns automatically
numeric_cols = df.select_dtypes(include=np.number).columns
print("\n Numeric Columns:", list(numeric_cols))

# ◇ Step 3: Handle Missing Values (mean filling for numeric)
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())

# ◇ Step 4: Remove outliers using IQR
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower) & (df[col] <= upper)]

# ◇ Step 5: Filter rows based on a column (optional)
# 📄 Change 'Salary' to your column name
if 'Salary' in df.columns:
    filtered = df[df['Salary'] > 60000]
    print("\n Filtered (Salary > 60000):\n", filtered)

# ◇ Step 6: Sort DataFrame based on a column
# 📄 Change 'Salary' to your sorting column
if 'Salary' in df.columns:
    sorted_df = df.sort_values(by='Salary', ascending=False) # descending order
    print("\n Sorted by Salary (High to Low):\n", sorted_df)
```

```

# ◇ Step 7: Group by column and get mean
# 📁 Change 'Country' to your grouping column
if 'Country' in df.columns:
    grouped = df.groupby('Country')[numeric_cols].mean()
    print("\n Grouped by Country:\n", grouped)

# ◇ Step 8: Save cleaned data
df.to_csv("cleaned_data.csv", index=False)
print("\n Cleaned data saved to 'cleaned_data.csv'")

# ◇ Step 9: Histogram of a column
# 📁 Change 'Salary' to any column
if 'Salary' in df.columns:
    df['Salary'].hist(bins=10)
    plt.title("Salary Distribution")
    plt.xlabel("Salary")
    plt.ylabel("Count")
    plt.show()

# ◇ Step 10: Correlation Heatmap
if len(numeric_cols) > 1:
    plt.figure(figsize=(8, 6))
    sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm')
    plt.title("Correlation Heatmap")
    plt.show()

```

2) Feature Scaling

A] Feature Scaling Boston Dataset

```

import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
# Load the dataset (California Housing as a modern replacement for Boston Housing)
data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
# Add a synthetic categorical column for demonstration (since the dataset is fully numerical)
df['house_category'] = np.random.choice(['Low', 'Medium', 'High'], size=len(df))
# 1 Feature Scaling - Standardization (Z-score Normalization)
scaler_standard = StandardScaler()
df_standardized = pd.DataFrame(scaler_standard.fit_transform(df.iloc[:, :-1]),
                                columns=df.columns[:-1])
# 2 Feature Scaling - Min-Max Normalization

```

```

scaler_minmax = MinMaxScaler()
df_normalized = pd.DataFrame(scaler_minmax.fit_transform(df.iloc[:, :-1]), columns=df.columns[:-1])
# 3 Feature Dummification - One-Hot Encoding using pandas
df_encoded = pd.get_dummies(df, columns=['house_category'], drop_first=True)
# Display Results
print("Standardized Data (First 5 rows):\n", df_standardized.head())
print("\nMin-Max Normalized Data (First 5 rows):\n", df_normalized.head())
print("\nOne-Hot Encoded Data (First 5 rows):\n", df_encoded.head())

```

B] Feature Scaling Iris Dataset.

```

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler, MinMaxScaler

# ◇ Load the Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# ◇ Add the target as a categorical column (instead of synthetic category)
df['species'] = iris.target # Species are numeric (0,1,2) here

# 📦 Optional: Convert numeric target to categorical labels
df['species'] = pd.Categorical.from_codes(iris.target, iris.target_names)

# 📊 Feature Scaling - Standardization (Z-score Normalization)
scaler_standard = StandardScaler()
numeric_cols = df.select_dtypes(include=np.number).columns # auto-detect numeric
df_standardized = pd.DataFrame(scaler_standard.fit_transform(df[numeric_cols]),
                                columns=numeric_cols)

# 📊 Feature Scaling - Min-Max Normalization
scaler_minmax = MinMaxScaler()
df_normalized = pd.DataFrame(scaler_minmax.fit_transform(df[numeric_cols]),
                                columns=numeric_cols)

# 📊 Feature Dummification - One-Hot Encoding using pandas
df_encoded = pd.get_dummies(df, columns=['species'], drop_first=True)

# 🔍 Display Results
print("✅ Standardized Data (First 5 rows):\n", df_standardized.head())
print("\n✅ Min-Max Normalized Data (First 5 rows):\n", df_normalized.head())
print("\n✅ One-Hot Encoded Data (First 5 rows):\n", df_encoded.head())

```

3) Hypothesis Testing

A] One-Sampled Test

```
import scipy.stats as stats
import numpy as np
# Step 1: Formulate Hypotheses
# Null Hypothesis (H0): The sample mean is equal to the population mean
# Alternative Hypothesis (H1): The sample mean is different from the population mean
population_mean = 50 # Assume known population mean
sample_data = [45, 55, 52, 48, 49, 51, 53, 47, 54, 50] # Sample dataset
# Step 2: Conduct Hypothesis Test (One-Sample t-test)
t_stat, p_value = stats.ttest_1samp(sample_data, population_mean)
# Step 3: Interpret the results
alpha = 0.05 # Significance level
print(f"Test Statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")
if p_value < alpha:
    print("Reject the null hypothesis: The sample mean is significantly different from the population mean.")
else:
    print("Fail to reject the null hypothesis: No significant difference between sample mean and population mean.")
```

B] Chi-Square Test

```
import numpy as np
import pandas as pd
from scipy.stats import chi2_contingency

# Step 1: Raw data for aptitude and job proficiency
aptitude = [85, 65, 50, 68, 87, 74, 65, 96, 68, 94, 73, 84, 85, 87, 91]
jobprof = [70, 90, 80, 89, 88, 86, 78, 67, 86, 90, 92, 94, 99, 93, 87]

# Step 2: Convert continuous scores to categorical levels
aptitude_cat = pd.cut(aptitude, bins=[0, 70, 85, 100], labels=["Low", "Medium", "High"])
jobprof_cat = pd.cut(jobprof, bins=[0, 80, 90, 100], labels=["Low", "Medium", "High"])

# Step 3: Create a contingency table
observed = pd.crosstab(aptitude_cat, jobprof_cat)

# Step 4: Perform Chi-Square Test
chi2_stat, p_value, dof, expected = chi2_contingency(observed)

# Step 5: Display results
```

```

print("\n**Chi-Square Test Results**")
print(f"Chi-Square Statistic: {chi2_stat:.4f}")
print(f"P-Value: {p_value:.4f}")
print(f"Degrees of Freedom: {dof}")
print("Expected Frequencies:\n", expected)

# Step 6: Decision Rule
alpha = 0.05 # Significance level
if p_value < alpha:
    print("Conclusion: Reject H0 → Aptitude and Job Proficiency are dependent.")
else:
    print("Conclusion: Fail to reject H0 → No significant relationship between Aptitude and Job Proficiency.")

```

4) ANOVA

```

from scipy.stats import f_oneway
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# Sample data for 3 groups
group1 = [85, 88, 90, 92, 87]
group2 = [78, 75, 80, 82, 76]
group3 = [90, 91, 89, 94, 92]

# ANOVA
f_stat, p_val = f_oneway(group1, group2, group3)
print("F-statistic:", round(f_stat, 2))
print("P-value:", round(p_val, 4))

if p_val < 0.05:
    print("Reject Ho: Significant difference found. Running post-hoc test...")
    data = group1 + group2 + group3
    labels = ['Group1']*len(group1) + ['Group2']*len(group2) + ['Group3']*len(group3)
    tukey = pairwise_tukeyhsd(endog=data, groups=labels, alpha=0.05)
    print(tukey)
else:
    print("Filed to Reject Ho: No significant difference found.")

```

5)Linear Regression

A] For Custom Data

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Dataset: Years of Experience and Salary
data = {
    'Years of Experience': [2, 10, 4, 20, 8, 12, 22],
    'Salary': [30000, 95000, 45000, 178000, 84000, 120000, 200000]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Define the independent variable (X) and dependent variable (y)
X = df[['Years of Experience']] # Independent variable
y = df['Salary'] # Dependent variable

# Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create a linear regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Output results
print(f"Mean Squared Error: {mse:.4f}")
print(f"R-squared: {r2:.4f}")

# Plotting the regression line
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', label='Regression line')
```

```
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.title('Linear Regression: Years of Experience vs Salary')
plt.legend()
plt.show()
```

B] Linear Regression for IRIS Dataset.

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target variable to the DataFrame (species)
df['species'] = iris.target

# Independent variable (petal.length)
X = df[['petal length (cm)']] # Independent variable

# Dependent variable (petal.width)
y = df['petal width (cm)'] # Dependent variable

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the linear regression model
model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Output results
print(f"Mean Squared Error: {mse:.4f}")
print(f"R-squared: {r2:.4f}")
```

```

# Plot the regression line
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', label='Regression line')
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('Linear Regression: Petal Length vs Petal Width')
plt.legend()
plt.show()

```

C] Multiple Linear Regression for IRIS Dataset.

```

import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Step 1: Load the Iris dataset
iris = load_iris()

# Create a DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target variable to the DataFrame (species)
df['Species'] = iris.target

# Step 2: Select features (independent variables) and target (dependent variable)
X = df.drop(columns=['Species']) # All features except 'Species'
y = df['petal width (cm)'] # Target variable (Petal Width)

# Step 3: Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Create and train the model (Multiple Linear Regression)
model = LinearRegression()
model.fit(X_train, y_train)

# Step 5: Make predictions
y_pred = model.predict(X_test)

# Step 6: Evaluate the model's performance using regression metrics
mae = mean_absolute_error(y_test, y_pred) # Mean Absolute Error

```

```

mse = mean_squared_error(y_test, y_pred) # Mean Squared Error
rmse = np.sqrt(mse) # Root Mean Squared Error
r2 = r2_score(y_test, y_pred) # R-squared (goodness of fit)

# Output results
print(f"Mean Absolute Error: {mae:.4f}")
print(f"Mean Squared Error: {mse:.4f}")
print(f"Root Mean Squared Error: {rmse:.4f}")
print(f"R-squared: {r2:.4f}")

# Step 7: Visualize Actual vs Predicted values
plt.scatter(y_test, y_pred, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--') # Ideal line
plt.xlabel('Actual Petal Width')
plt.ylabel('Predicted Petal Width')
plt.title('Actual vs Predicted Petal Width')
plt.show()

```

6) Logistic Regression.

A] Logistic Regression for Iris Data. [Binary Classification]

```

import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Step 1: Load the Iris dataset
iris = load_iris()

# Create a DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Add target variable (species)
df['Species'] = iris.target

# Step 2: Convert to binary classification (Class 0 vs Class 1)
# Select only class 0 and class 1 for binary classification
df_binary = df[df['Species'] < 2]

# Step 3: Define features (X) and target (y)
X = df_binary.drop(columns=['Species']) # Features

```

```

y = df_binary['Species'] # Target (Binary outcome: Class 0 or Class 1)

# Step 4: Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 5: Create and train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 6: Make predictions on the test set
y_pred = model.predict(X_test)

# Step 7: Evaluate the model using classification metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

# Output the metrics
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")

# Step 8: Confusion Matrix
print("\nConfusion Matrix:")
print(conf_matrix)

```

B] Logistic Regression for Custom Data.

```

import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
confusion_matrix

# Step 1: Load your dataset
data = {
    'Feature1': [2, 3, 5, 7, 9, 11, 13, 15, 17, 19],
    'Feature2': [1, 2, 2, 3, 3, 4, 4, 5, 5, 6],
    'Label': [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)

```

```

# Step 2: Define features and target
X = df[['Feature1', 'Feature2']] # Independent variables
y = df['Label']                 # Binary target variable

# Step 3: Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Step 4: Train the Logistic Regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 5: Predict and evaluate
y_pred = model.predict(X_test)

# Evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

# Step 6: Output
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 Score: {f1:.4f}")
print("\nConfusion Matrix:")
print(conf_matrix)

```

7) Decision Tree

A] For Titanic Dataset.

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import seaborn as sns

# Load and preprocess Titanic dataset
df = sns.load_dataset('titanic')[['survived', 'pclass', 'sex', 'age', 'fare']].dropna()
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

```

```

# Features and target
X = df[['pclass', 'sex', 'age', 'fare']]
y = df['survived']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.2f}")

# Show decision rules
print("\nDecision Tree Rules:")
print(export_text(model, feature_names=X.columns.tolist()))

```

B] For IRIS Dataset.

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Features and target
X = df[iris.feature_names]
y = df['species']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model

```

```

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation
print(f"Accuracy : {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred, average='weighted'):.2f}")
print(f"Recall : {recall_score(y_test, y_pred, average='weighted'):.2f}")
print(f"F1 Score : {f1_score(y_test, y_pred, average='weighted'):.2f}")

# Show decision rules
print("\nDecision Tree Rules:")
print(export_text(model, feature_names=iris.feature_names))

```

C] For custom Dataset

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Custom data
data = {
    'Feature1': [2, 3, 5, 7, 9, 11, 13, 15, 17, 19],
    'Feature2': [1, 2, 2, 3, 3, 4, 4, 5, 5, 6],
    'Label': [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
}

# Create DataFrame
df = pd.DataFrame(data)
X = df[['Feature1', 'Feature2']]
y = df['Label']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation

```

```

print(f"Accuracy : {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_pred):.2f}")

# Show decision rules
print("\nDecision Tree Rules:")
print(export_text(model, feature_names=X.columns.tolist()))

```

8)KMeans

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

# Generate custom dataset
data = {'Feature1': [1, 2, 3, 8, 9, 10, 15, 16, 17],
        'Feature2': [2, 3, 4, 7, 8, 9, 12, 13, 14],
        'Feature3': [5, 6, 7, 10, 11, 12, 15, 16, 18],
        'Feature4': [1, 2, 1, 4, 5, 6, 7, 8, 9]
        }

df = pd.DataFrame(data)

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Elbow Method to find optimal number of clusters
inertia = []
for k in range(1, 6):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plot Elbow Method
plt.plot(range(1, 6), inertia, marker='o')
plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.show()

# Choose k=3 from Elbow plot and apply K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)

```

```

# Calculate Silhouette Score
sil_score = silhouette_score(X_scaled, df['Cluster'])
print(f'Silhouette Score: {sil_score:.4f}')

# Visualize Clusters
plt.scatter(df['Feature1'], df['Feature2'], c=df['Cluster'], cmap='viridis')
plt.xlabel('Feature1')
plt.ylabel('Feature2')
plt.title('K-Means Clustering Results')
plt.show()

# Analyze Cluster Characteristics (mean of features per cluster)
print("\nCluster Characteristics (mean values):")
print(df.groupby('Cluster').mean())

```

9) PCA for dimensionality Reduction

A] PCA for IRIS dataset.

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Plot explained variance ratio
plt.plot(range(1, len(pca.explained_variance_ratio_) + 1), pca.explained_variance_ratio_, marker='o')
plt.title('Explained Variance Ratio for Iris Dataset')
plt.xlabel('Principal Components')
plt.ylabel('Variance Ratio')
plt.show()

# Cumulative explained variance

```

```
plt.plot(range(1, len(pca.explained_variance_ratio_) + 1), pca.explained_variance_ratio_.cumsum(),
marker='o')
plt.title('Cumulative Explained Variance for Iris Dataset')
plt.xlabel('Principal Components')
plt.ylabel('Cumulative Variance')
plt.show()

# 2D Visualization using first two principal components
X_pca_2D = X_pca[:, :2]
plt.scatter(X_pca_2D[:, 0], X_pca_2D[:, 1], c=iris.target, cmap='viridis')
plt.title('PCA 2D Visualization for Iris Dataset')
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.colorbar(label='Species')
plt.show()
```

B] For Custom Data

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Generate custom dataset
data = {'Feature1': [1, 2, 3, 8, 9, 10, 15, 16, 17],
        'Feature2': [2, 3, 4, 7, 8, 9, 12, 13, 14],
        'Feature3': [5, 6, 7, 10, 11, 12, 15, 16, 18],
        'Feature4': [1, 2, 1, 4, 5, 6, 7, 8, 9]}
df = pd.DataFrame(data)

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Plot explained variance ratio
plt.plot(range(1, len(pca.explained_variance_ratio_) + 1), pca.explained_variance_ratio_, marker='o')
plt.title('Explained Variance Ratio for Custom Dataset')
plt.xlabel('Principal Components')
plt.ylabel('Variance Ratio')
plt.show()
```

```

# Cumulative explained variance
plt.plot(range(1, len(pca.explained_variance_ratio_) + 1), pca.explained_variance_ratio_.cumsum(),
marker='o')
plt.title('Cumulative Explained Variance for Custom Dataset')
plt.xlabel('Principal Components')
plt.ylabel('Cumulative Variance')
plt.show()

# 2D Visualization using first two principal components
X_pca_2D = X_pca[:, :2]
plt.scatter(X_pca_2D[:, 0], X_pca_2D[:, 1], cmap='viridis') # Using 'cmap' for color
plt.title('PCA 2D Visualization for Custom Dataset')
plt.xlabel('PCA 1')
plt.ylabel('PCA 2')
plt.colorbar() # Adds color bar to the plot
plt.show()

```

10) Data Visualization and Storytelling.

A] Custom Data Visualization.

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Updated Custom Dataset
data = {
    'gender': ['Male', 'Female', 'Female', 'Male', 'Male'],
    'churn': [1, 0, 1, 0, 1],
    'age': [25, 30, 40, 35, 50],
    'service type': ['A', 'B', 'A', 'B', 'A'],
    'tenure': [12, 24, 36, 48, 60],
    'feature1': [5.1, 4.3, 6.2, 5.5, 4.8],
    'feature2': [2.1, 3.5, 1.8, 2.9, 3.2],
    'cluster': ['Group1', 'Group2', 'Group1', 'Group2', 'Group1']
}

df = pd.DataFrame(data)

# 1. Distribution of Features
sns.histplot(df['feature1'], kde=True, color='blue', label='Feature1')
sns.histplot(df['feature2'], kde=True, color='green', label='Feature2')
sns.histplot(df['age'], kde=True, color='red', label='Age')
plt.legend(title='Features')

```

```

plt.title('Feature Distributions')
plt.show()

# 2. Pairplot (relationships)
sns.pairplot(df[['feature1', 'feature2', 'age', 'tenure', 'cluster']], hue='cluster', palette='Set2')
plt.suptitle('Pairwise Relationships by Cluster', y=1.02)
plt.show()

# 3. Correlation Heatmap
sns.heatmap(df[['feature1', 'feature2', 'age', 'tenure', 'churn']].corr(), annot=True, cmap='coolwarm',
fmt='.2f')
plt.title('Feature Correlation')
plt.show()

# 4. Box Plot (Feature1 by Service Type)
sns.boxplot(x='service type', y='feature1', data=df, hue='service type')
plt.title('Box Plot of Feature1 by Service Type')
plt.show()

# 5. Violin Plot (Feature2 by Gender)
sns.violinplot(x='gender', y='feature2', data=df, inner='quart', hue='gender')
plt.title('Violin Plot of Feature2 by Gender')
plt.show()

```

B] IRIS Data Visualization.

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris dataset
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target_names[iris.target]

# 1. Distribution of Features (Histograms)
sns.histplot(df['sepal length (cm)'], kde=True, color='blue', label='Sepal Length')
sns.histplot(df['sepal width (cm)'], kde=True, color='green', label='Sepal Width')
sns.histplot(df['petal length (cm)'], kde=True, color='red', label='Petal Length')
sns.histplot(df['petal width (cm)'], kde=True, color='orange', label='Petal Width')
plt.legend(title='Features')
plt.title('Feature Distributions')
plt.show()

```

```

# 2. Pairplot (Relationships between features)
sns.pairplot(df, hue='species', palette='Set1')
plt.suptitle('Pairwise Relationships', y=1.02)
plt.show()

# 3. Correlation Heatmap (excluding 'species' column)
sns.heatmap(df.drop(columns='species').corr(), annot=True, cmap='coolwarm', fmt='.2f',
linewidths=1)
plt.title('Feature Correlation')
plt.show()

# 4. Box Plot (Sepal Length by Species)
sns.boxplot(x='species', y='sepal length (cm)', data=df, hue='species')
plt.title('Box Plot of Sepal Length by Species')
plt.show()

# 5. Violin Plot (Petal Length by Species)
sns.violinplot(x='species', y='petal length (cm)', data=df, inner='quart', hue='species')
plt.title('Violin Plot of Petal Length by Species')
plt.show()

```

11) Excel Important formulas:

✓ 1. Conditional Formatting

Apply formatting based on specific conditions (e.g., highlight sales > 1000 or churned customers).

Steps:

1. Select the data range (e.g., B2:B100).
2. Go to **Home > Conditional Formatting > New Rule**.
3. Choose **“Format only cells that contain”** or use a **formula**.

✓ 2. Create a Pivot Table

Used to **summarize large data** (e.g., total sales by region).

Steps:

1. Select your dataset.
2. Go to **Insert > Pivot Table**.
3. Choose the location (new worksheet or same sheet).

4. Drag fields into:

- **Rows** (e.g., Region)
 - **Values** (e.g., Sales)
 - **Filters** (optional, e.g., Year)
-

✓ 3. Use VLOOKUP() to Retrieve Info from Another Table

Fetch data from another sheet or table using a common key (e.g., customer ID).

Formula Syntax:

=VLOOKUP(lookup_value, table_array, col_index_num, [range_lookup])

Example:

To get customer name using customer ID from Sheet2:

=VLOOKUP(A2, Sheet2!A:B, 2, FALSE)

- A2 → Lookup value
 - Sheet2!A:B → Table to search
 - 2 → Column number from which to return the result
 - FALSE → Exact match
-

✓ 4. What-If Analysis Using Goal Seek

Used to find the input needed to achieve a desired result.

Steps:

1. Suppose Cell B5 contains formula: =B1*B2 (e.g., Price × Quantity = Total).
2. Go to **Data > What-If Analysis > Goal Seek**.
3. Set:
 - **Set Cell:** B5 (the result cell)
 - **To Value:** e.g., 1000 (desired result)
 - **By Changing Cell:** B2 (e.g., quantity)
4. Click OK to see the result.